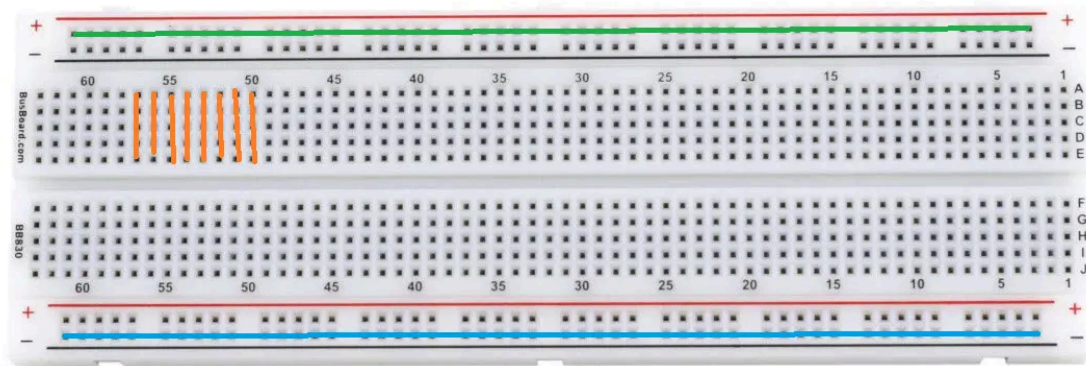


Arduino

Breadboard

-Breadboard จะมีการจัด핀ตามแนว แนวนอน และ แนวตั้ง



- แถบด้านบนกับด้านล่าง เป็นเส้นตรงแนวนอนทั้งหมด (มักจะมีแถบสีแดงหรือสีน้ำเงิน) ใช้สำหรับเชื่อมต่อกับ แหล่งจ่ายไฟบวก (VCC) และ แหล่งจ่ายไฟลบ (GND) ข้อสำคัญคือ ถ้าต่อไฟที่จุดใดจุดหนึ่งในแถบนี้นั้น แถบจะเชื่อมต่อกันเป็น แนวนอน จากภาพ
- ส่วนของพื้นที่ด้านในจะมีการเชื่อมต่อเป็น แนวตั้ง จากภาพ ซึ่งในแต่ละคอลัมน์ที่มีรูหลายรูเรียงตามแนวตั้งจะเชื่อมต่อกัน โดยคอลัมน์เหล่านี้ใช้สำหรับเชื่อมต่ออุปกรณ์ต่าง ๆ ในวงจร

ไฟกระพริบ

สร้างไฟกระพริบโดยให้กำหนดความถี่ในการกระพริบเอง
LED แบบ 2 ขา



- ขา Anode (ขายาว) เป็นขาไฟบวก ต้องต่อกับไฟบวก

-ขา Cathode (ขาสั้น) เป็นขาไฟลบ ต่อกับไฟลบหรือผ่านตัวต้านทาน

```
// Define the LED pin

int ledPin = 13;

// Setup function: runs once

void setup() {

    // Initialize the LED pin as an OUTPUT

    pinMode(ledPin, OUTPUT);

}

// Loop function: runs repeatedly

void loop() {

    // Turn the LED ON

    digitalWrite(ledPin, HIGH);

    // Wait for 1 second

    delay(1000);

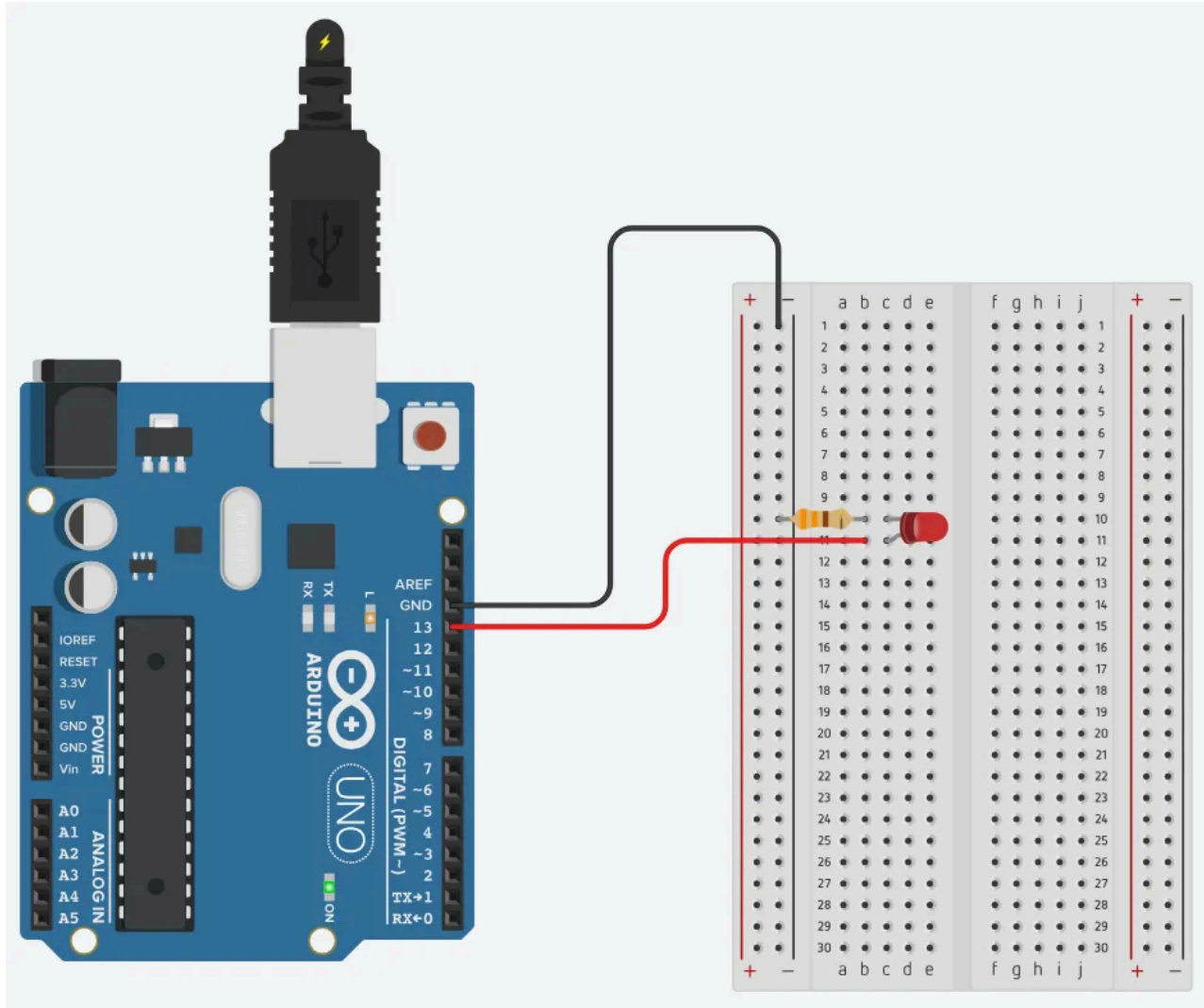
    // Turn the LED OFF

    digitalWrite(ledPin, LOW);

    // Wait for another second

    delay(1000);

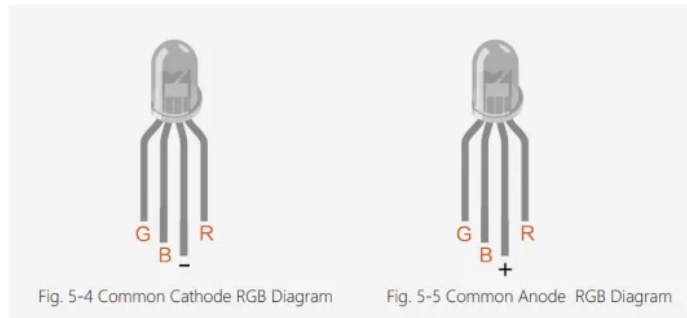
}
```



RGB

เขียนโปรแกรมแสดงสี 7 สี โดยเว้นช่วงสีละ 1 วินาที

LED แบบ 4 ขา



Common Cathode

Common Anode

-ขา Red (R): ขาที่เชื่อมต่อกับแสงสีแดง

-ขา Green (G): ขาที่เชื่อมต่อกับแสงสีเขียว

-ขา Blue (B): ขาที่เชื่อมต่อกับแสงสีน้ำเงิน

-ขา Common (C): ขาอาจจะเป็นขา Common Anode (CA) หรือ Common Cathode (CC)

Common Anode (CA): ขา Common จะเชื่อมต่อกับแหล่งจ่ายไฟบวก (VCC) และขาอื่น ๆ จะต้องต่อไปยัง GND เพื่อให้แสงสว่าง ค่า HIGH กับ LOW สลับกัน (HIGH ปิดไฟ, LOW เปิดไฟ)

Common Cathode (CC): ขา Common จะเชื่อมต่อกับ GND และขาอื่น ๆ จะต้องต่อไปยัง VCC เพื่อให้แสงสว่าง

-RGB ที่แจกคือประเภท Common Anode

```
// Define the LED pin
```

```
int ledPin9 = 9;
```

```
int ledPin10 = 10;
```

```
int ledPin11 = 11;
```

```
// Setup function: runs once
```

```
void setup() {
```

```
// Initialize the LED pin as an OUTPUT

pinMode(ledPin9, OUTPUT);

pinMode(ledPin10, OUTPUT);

pinMode(ledPin11, OUTPUT);

}

// Loop function: runs repeatedly

void loop() {

    // Turn the LED ON

    digitalWrite(ledPin9, HIGH);

    // Wait for 1 second

    delay(1000);

    // Turn the LED OFF

    digitalWrite(ledPin9, LOW);

    // Wait for another second

    delay(1000);

    // Turn the LED ON

    digitalWrite(ledPin10, HIGH);

    // Wait for 1 second

    delay(1000);

    // Turn the LED OFF

    digitalWrite(ledPin10, LOW);

    // Wait for another second

    delay(1000);

    // Turn the LED ON
```

```

digitalWrite(ledPin11, HIGH);

// Wait for 1 second

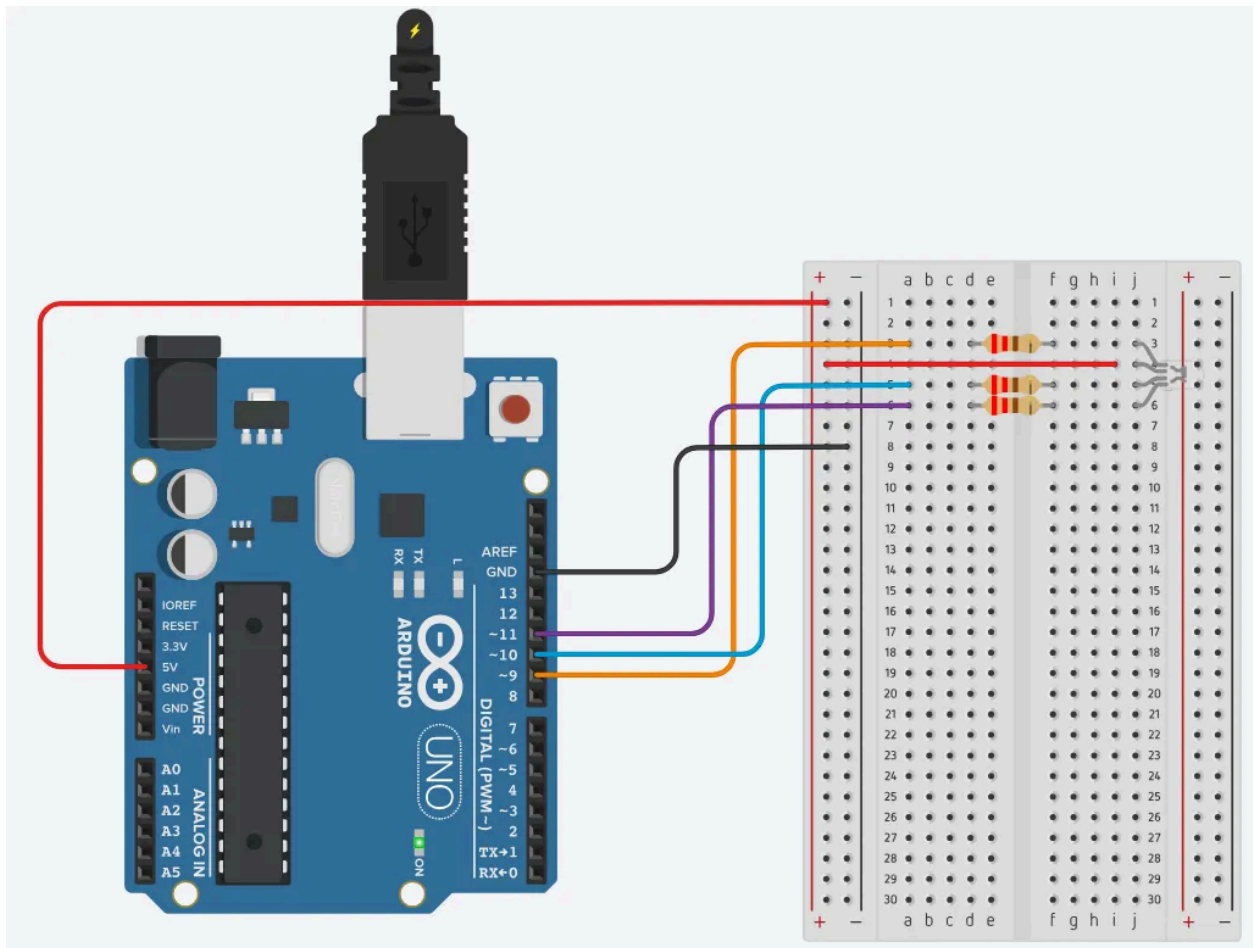
delay(1000);

// Turn the LED OFF

digitalWrite(ledPin11, LOW);

}

```

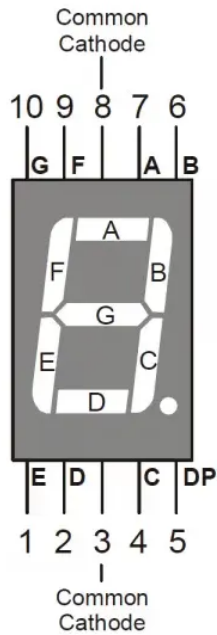


-GND ไม่ต้องต่อก็ได้

7Segment (1 digit)

เขียนโปรแกรมแสดงเลข 0-9 โดยเว้นช่วงละ 1 วินาที

-ระบุตำแหน่งขาของ 7-segment



-การเชื่อมต่อ Pin กับ 7-segment

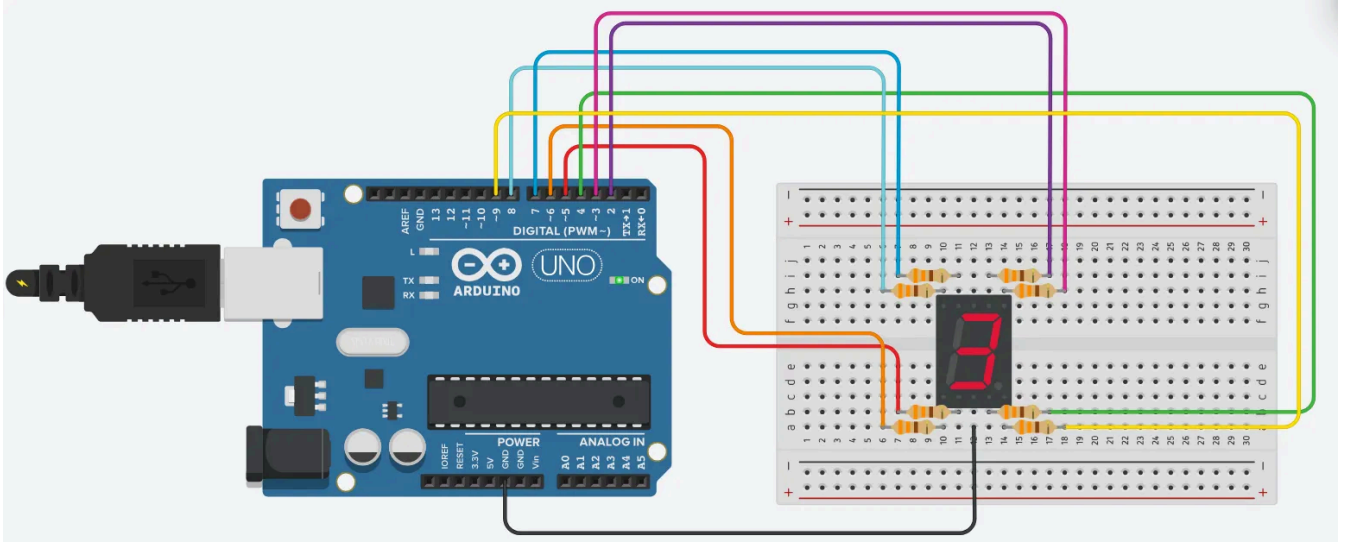
- ขา E (ขา 1) → Digital Pin 6
- ขา D (ขา 2) → Digital Pin 5
- ขา Common (ขา 3) → GND (ถ้าเป็น Common Cathode)
หรือ +5V (ถ้าเป็น Common Anode)
- ขา C (ขา 4) → Digital Pin 4
- ขา DP (ขา 5) → Digital Pin 9
- ขา B (ขา 6) → Digital Pin 3
- ขา A (ขา 7) → Digital Pin 2
- ขา F (ขา 9) → Digital Pin 7
- ขา G (ขา 10) → Digital Pin 8

```
int num_array[10][7] = { { 1,1,1,1,1,1,0 }, // 0
                          { 0,1,1,0,0,0,0 }, // 1
                          { 1,1,0,1,1,0,1 }, // 2
                          { 1,1,1,1,0,0,1 }, // 3
                          { 0,1,1,0,0,1,1 }, // 4
                          { 1,0,1,1,0,1,1 }, // 5
                          { 1,0,1,1,1,1,1 }, // 6
                          { 1,1,1,0,0,0,0 }, // 7
                          { 1,1,1,1,1,1,1 }, // 8
                          { 1,1,1,0,0,1,1 }
                        }; // 9
```

```

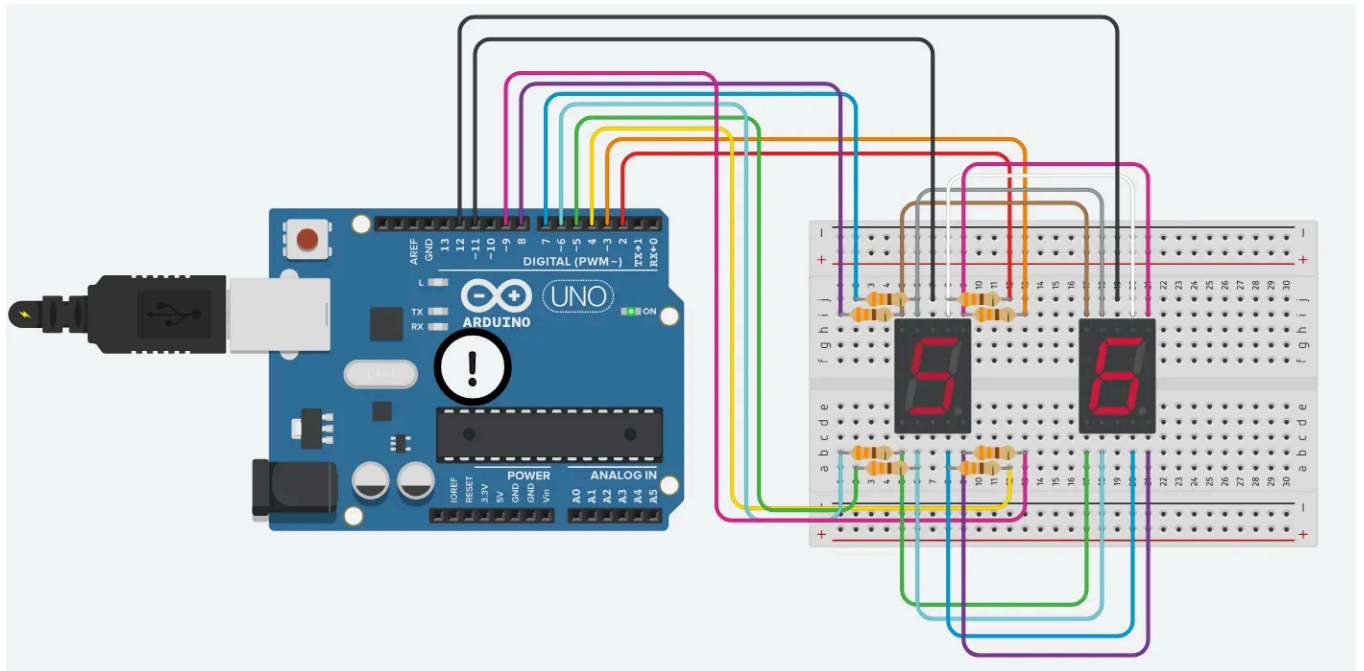
// Define the pins for each segment of the 7-segment display
const int segment_a = 2;
const int segment_b = 3;
const int segment_c = 4;
const int segment_d = 5;
const int segment_e = 6;
const int segment_f = 7;
const int segment_g = 8;
void setup() {
    // set pin modes for each segment
    pinMode(segment_a, OUTPUT);
    pinMode(segment_b, OUTPUT);
    pinMode(segment_c, OUTPUT);
    pinMode(segment_d, OUTPUT);
    pinMode(segment_e, OUTPUT);
    pinMode(segment_f, OUTPUT);
    pinMode(segment_g, OUTPUT);
}
void loop() {
    // Loop through each digit (0-9)
    for (int digit = 0; digit < 10; digit++) {
        displayDigit(digit);
        delay(1000); // Wait for 1 second before displaying the next digit
    }
}
// Function to display a specific digit on the 7-segment display
void displayDigit(int digit) {
    // Set each segment according to the num_array for the given digit
    digitalWrite(segment_a, num_array[digit][0]);
    digitalWrite(segment_b, num_array[digit][1]);
    digitalWrite(segment_c, num_array[digit][2]);
    digitalWrite(segment_d, num_array[digit][3]);
    digitalWrite(segment_e, num_array[digit][4]);
    digitalWrite(segment_f, num_array[digit][5]);
    digitalWrite(segment_g, num_array[digit][6]);
}

```

7Segment (2 digit)

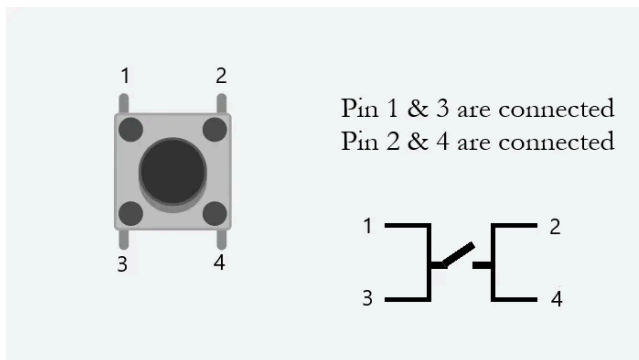
เขียนโปรแกรมแสดงเลขที่ละ 2 หลักท้ายของรหัสนักศึกษา เช่นรหัส 66070012 ก็แสดง 64 07 00 12 ช่วง
ละ 1 วินาที



Color LED Switch

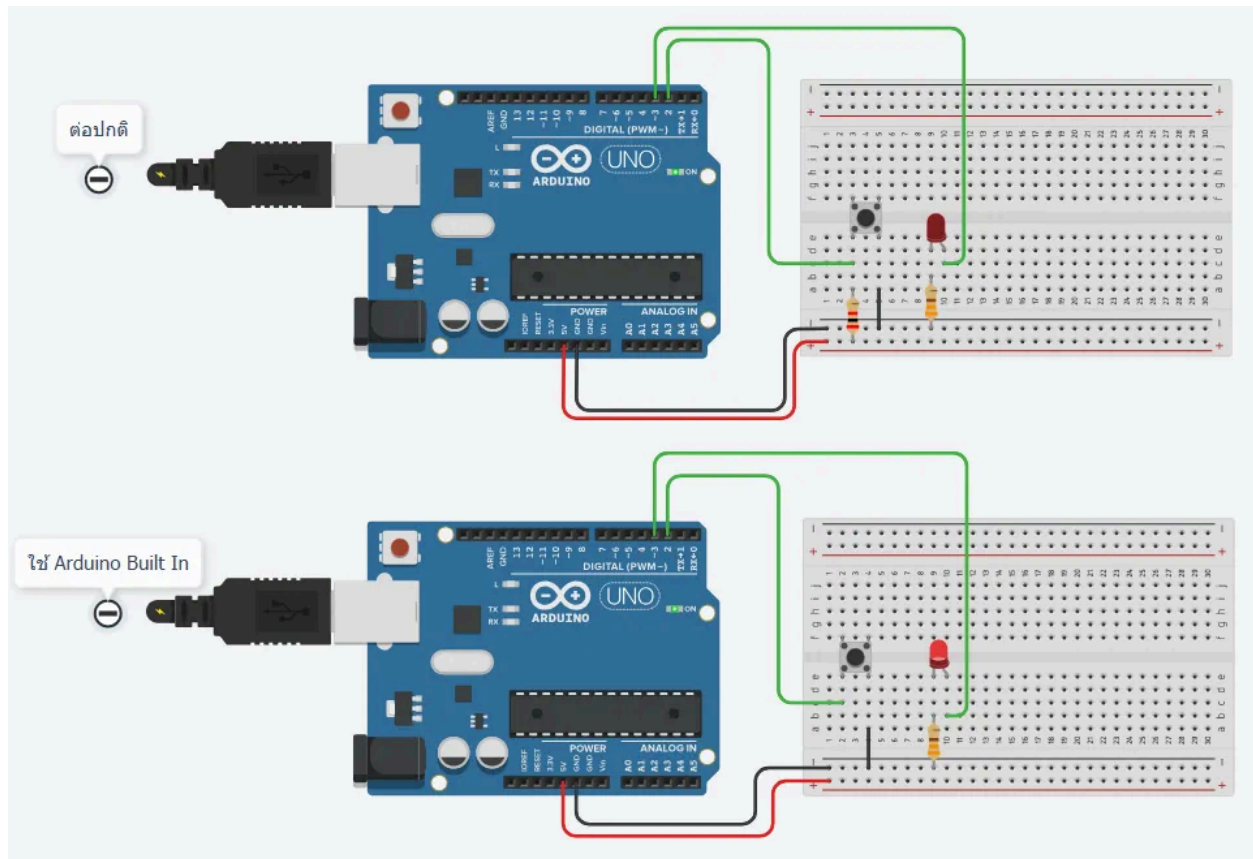
-Push Button เป็นอุปกรณ์อิเล็กทรอนิกส์ที่ใช้เพื่อควบคุมการไหลของกระแสไฟฟ้าโดยการกดปุ่ม เมื่อไม่ได้กดปุ่มจะไม่มีกระแสไฟฟ้าไหลผ่านแต่เมื่อกดปุ่มกระแสไฟฟ้าจะไหลผ่าน

-มีสี่ขาโดยขา 1 และ 3 เชื่อมต่อกันและขา 2 และ 4 เชื่อมต่อกัน



Pull-Up และ Pull-Down Resistors

-Pull-Up Resistor: คือการกำหนดสัญญาณ 1 ให้กับสวิตช์ตลอดเวลา เมื่อสวิตช์ถูกกดจะให้สัญญาณ 0



```
const int BTN = 2;
```

```
const int LED = 3;
```

```
bool is_led_on = true;
```

```
void setup()
```

```
{  
  pinMode(BTN, INPUT_PULLUP); // Use internal pull-up resistor  
  pinMode(LED, OUTPUT);  
}
```

```
void loop()
```

```
{  
  if (digitalRead(BTN) == LOW) {  
    is_led_on = !is_led_on;  
    delay(100);  
  }
```

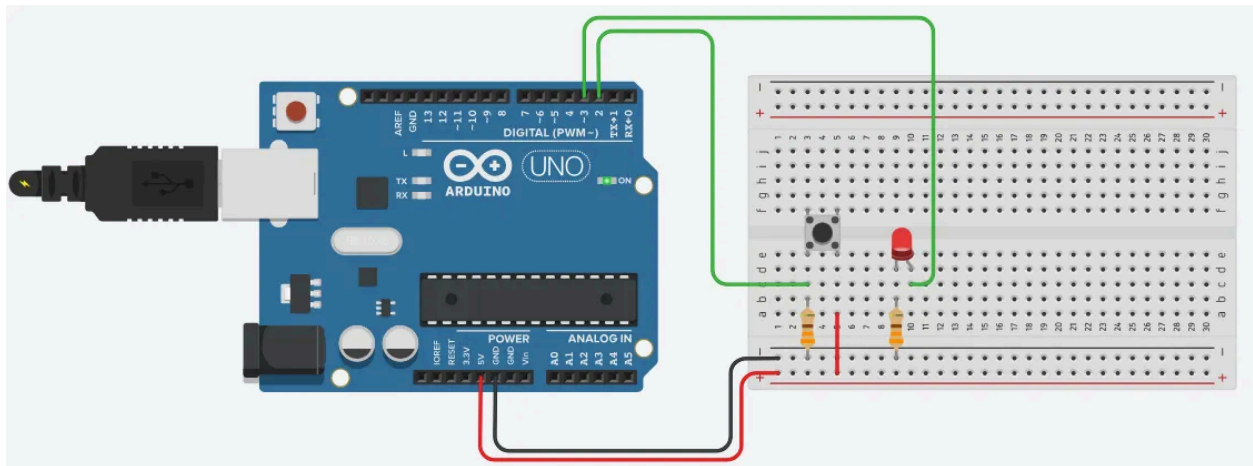
```
  if (is_led_on) {
```

```

    digitalWrite(LED, HIGH);
} else {
    digitalWrite(LED, LOW);
}
delay(5);
}

```

Pull-Down Resistor: การกำหนดสัญญาณ 0 ให้กับสวิตช์ตลอดเวลา เมื่อสวิตช์ถูกกดจะให้สัญญาณ 1



```

const int BTN = 2;
const int LED = 3;
bool is_led_on = true;

void setup()
{
    pinMode(BTN, INPUT);
    pinMode(LED, OUTPUT);
}

void loop()
{
    if (digitalRead(BTN) == HIGH) {
        is_led_on = !is_led_on;
        delay(100);
    }

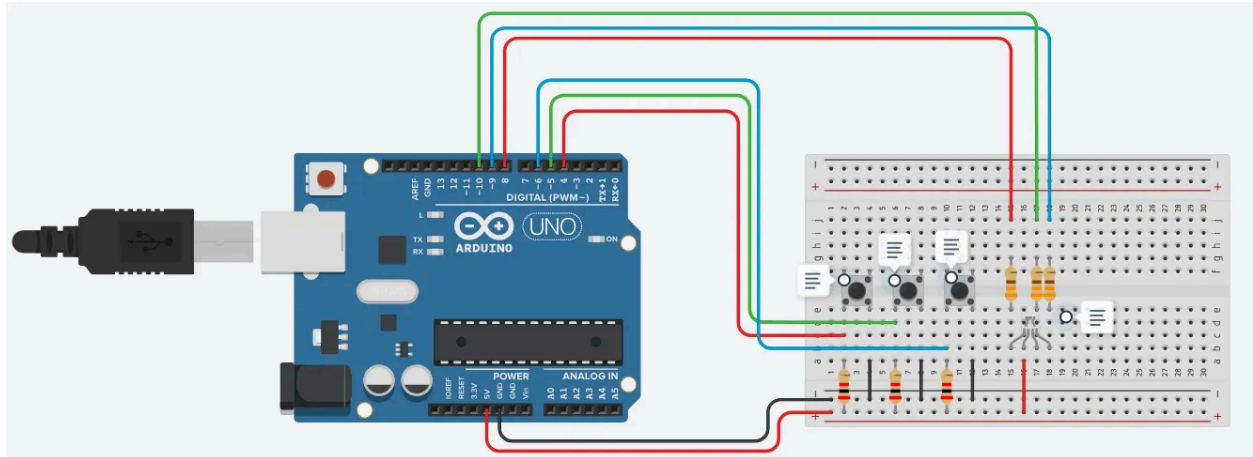
    if (is_led_on) {
        digitalWrite(LED, HIGH);
    } else {
        digitalWrite(LED, LOW);
    }
    delay(5);
}

```

Arduino ออกแบบให้สามารถทำงานแบบ Pull UP ได้แบบไม่ต้องใช้ตัวต้านทานเพิ่ม เพียงแค่พิมพ์โค้ดคำสั่งให้เป็นโหมดนี้

```
pinMode(ขา, INPUT_PULLUP);
```

ต่อ Switch จำนวน 3 ตัว เมื่อกด switch แต่ละตัว ให้ Color LED ติดคนละสีเช่น Red, Green , Blue และเมื่อปล่อย switch ให้ Color LED ดับ



```
const int BTN_RED = 4;  
const int BTN_GREEN = 5;  
const int BTN_BLUE = 6;
```

```
const int PIN_RED = 8;  
const int PIN_GREEN = 9;  
const int PIN_BLUE = 10;
```

```
bool is_red_on = false;  
bool is_green_on = false;  
bool is_blue_on = false;
```

```
void setup() {  
  pinMode(BTN_RED, INPUT);  
  pinMode(BTN_GREEN, INPUT);  
  pinMode(BTN_BLUE, INPUT);  
  
  pinMode(PIN_RED, OUTPUT);  
  pinMode(PIN_GREEN, OUTPUT);  
  pinMode(PIN_BLUE, OUTPUT);  
}
```

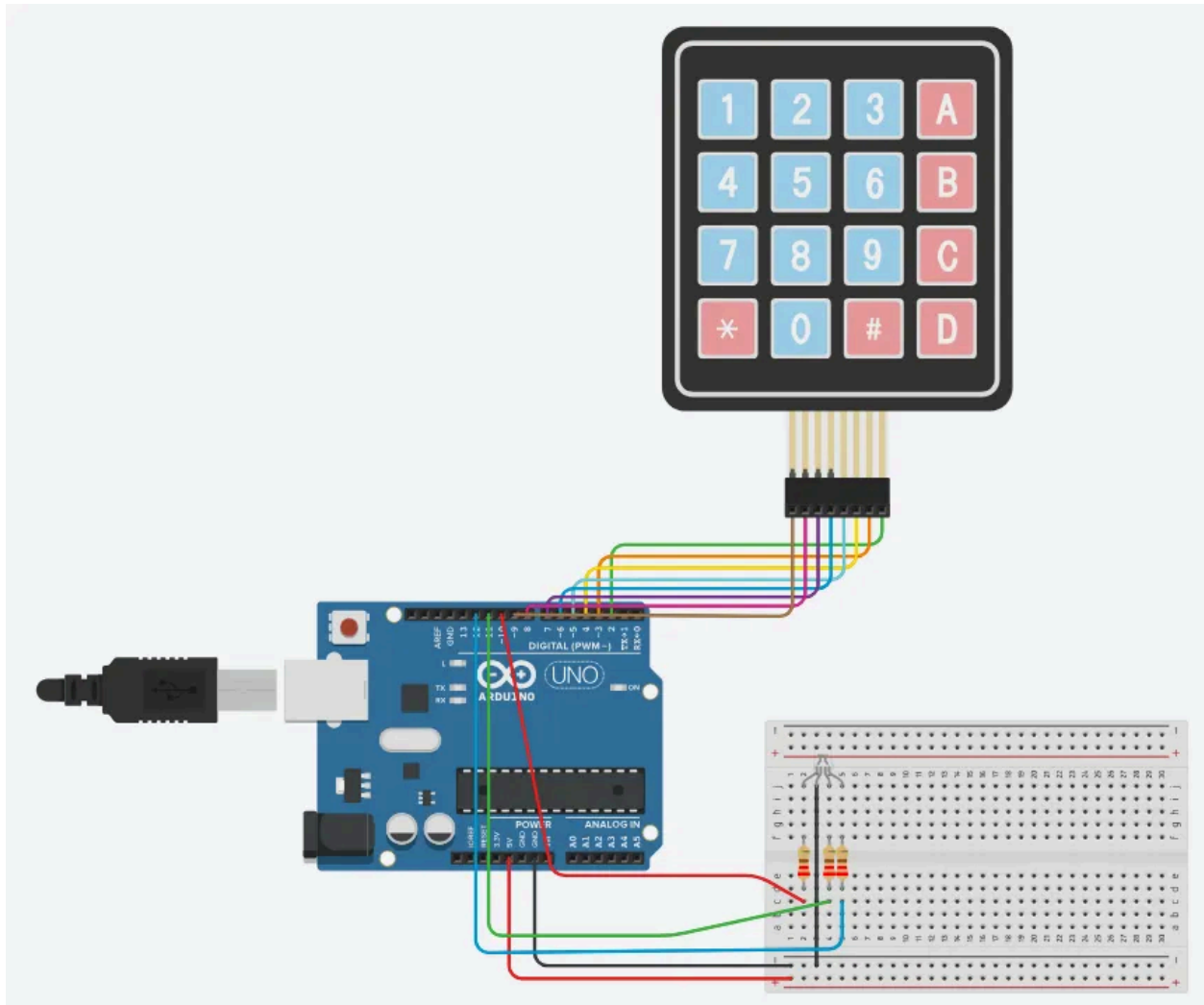
```
void loop() {  
  digitalWrite(PIN_RED, digitalRead(BTN_RED) == HIGH ? HIGH : LOW);  
  digitalWrite(PIN_GREEN, digitalRead(BTN_GREEN) == HIGH ? HIGH : LOW);  
  digitalWrite(PIN_BLUE, digitalRead(BTN_BLUE) == HIGH ? HIGH : LOW);  
}
```

```
}
```

```
void change_stat(int btn_pin, int led_pin, bool* status) {  
  if (digitalRead(btn_pin) == LOW) {  
    *status = !(*status);  
    if (*status) {  
      digitalWrite(led_pin, HIGH);  
    } else {  
      digitalWrite(led_pin, LOW);  
    }  
    delay(1000);  
  }  
}
```

Keypad (RGB)

เขียนโปรแกรมรับค่าจาก Keypad โดยให้แสดงสี ของ Color LED เป็นสีต่างๆ ตามปุ่มที่กด โดยกำหนดสีเองตามใจชอบ



```
#include <Keypad.h>
```

```
const byte numRows = 4; // Number of rows on the keypad  
const byte numCols = 4; // Number of columns on the keypad
```

```
const int LED_RED = 10;  
const int LED_GREEN = 11;  
const int LED_BLUE = 12;
```

```
bool is_red_on = false;  
bool is_green_on = false;  
bool is_blue_on = false;
```

```

// Keymap defines the key pressed according to the row and columns
char keymap[numRows][numCols] =
{
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

// Keypad connections to the Arduino pins
byte rowPins[numRows] = {9, 8, 7, 6}; // Rows 0 to 3
byte colPins[numCols] = {5, 4, 3, 2}; // Columns 0 to 3

// Initializes an instance of the Keypad class
Keypad myKeypad = Keypad(makeKeymap(keymap), rowPins, colPins, numRows, numCols);

void setup() {
    Serial.begin(9600);
    pinMode(LED_RED, OUTPUT);
    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_BLUE, OUTPUT);
}

void loop() {
    char keypressed = myKeypad.getKey();
    if (keypressed != NO_KEY) {
        Serial.println(keypressed);
        switch (keypressed) {
            case ":":
                is_red_on = !is_red_on;
                digitalWrite(LED_RED, is_red_on ? HIGH : LOW);
                break;

            ...

            default:
                break;
        }
    }
}

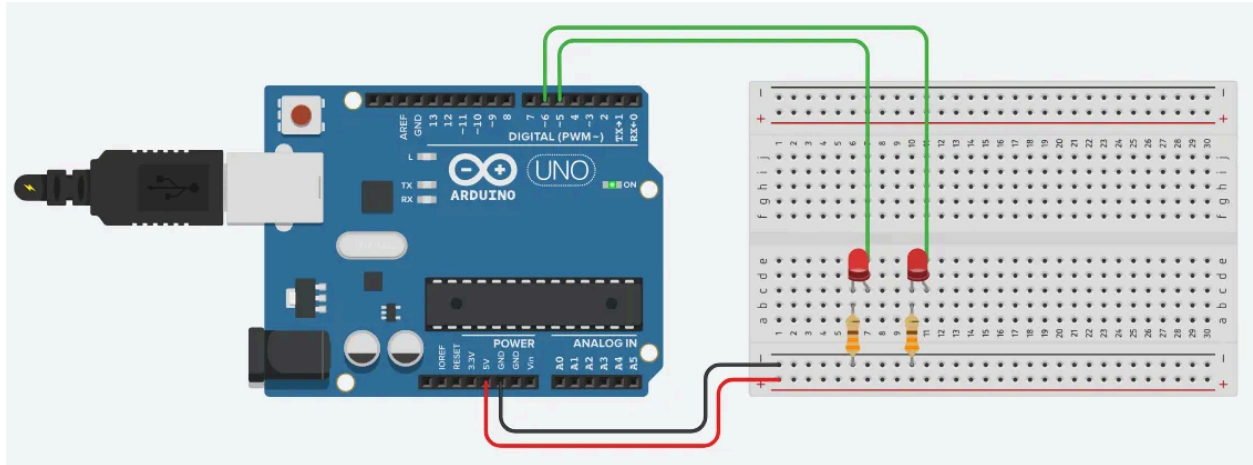
```


Analog LED Fading

เลือกใช้ PWM Pin อื่นๆ เป็นจำนวน 2 Pin

LED1 Fading จาก Off ไปยัง Full-Bright

LED2 Fading จาก Off ไปยัง Full-Bright เร็วเป็นสองเท่าของ LED1



```
const int ledPin1 = 5; // เลือก PWM Pin สำหรับ LED1
const int ledPin2 = 6; // เลือก PWM Pin สำหรับ LED2
int brightness1 = 0; // ความสว่างของ LED1
int brightness2 = 0; // ความสว่างของ LED2

void setup() {
  // กำหนด Pin เป็น Output
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
}

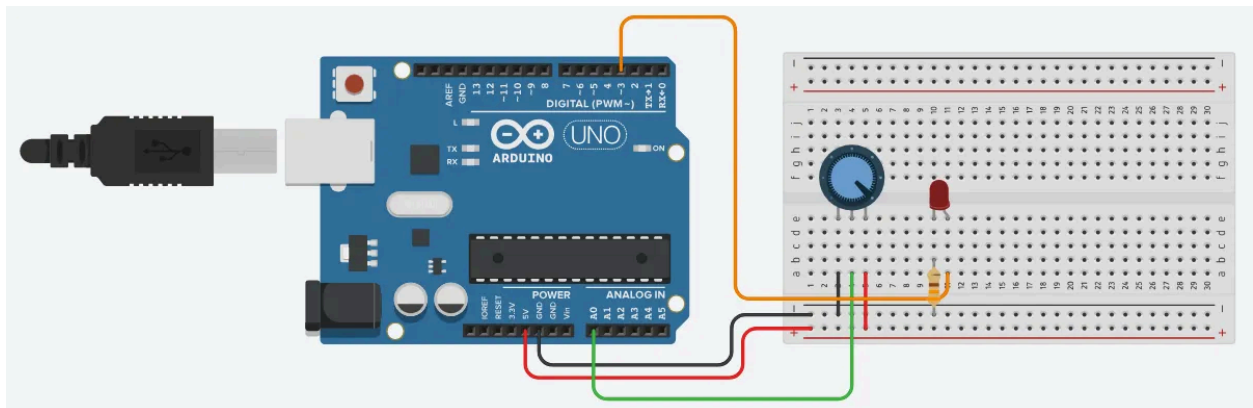
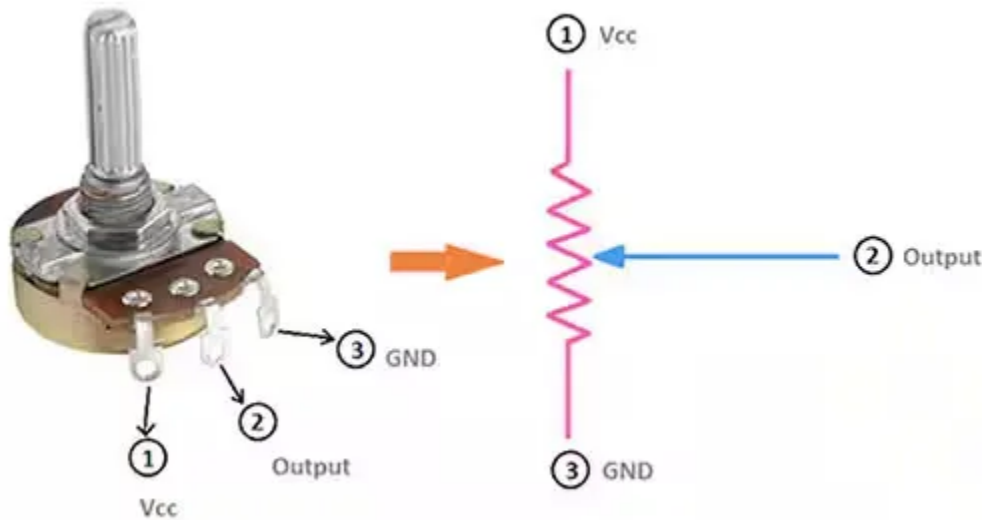
void loop() {
  // Fading LED1 จาก Off ไปยัง Full-Bright
  analogWrite(ledPin1, brightness1);
  brightness1++;
  // Fading LED2 จาก Off ไปยัง Full-Bright เร็วเป็นสองเท่า
  analogWrite(ledPin2, brightness2);
  brightness2 += 2;

  // ตรวจสอบความสว่าง
  if (brightness1 > 255) {
    brightness1 = 0;
  }
  if (brightness2 > 255) {
    brightness2 = 0;
  }

  delay(10); // หน่วงเวลาเล็กน้อยสำหรับการปรับความสว่าง
}
```

Analog Reading

เขียนโปรแกรม Arduino เพื่อควบคุมความสว่างของ LED โดยใช้ Potentiometer เพื่อปรับค่าความสว่าง LED ให้มีการเปลี่ยนแปลงตามค่าที่ Potentiometer ปรับ



```
const int ledPin = 3; // Pin for LED (PWM pin)
const int potPin = A0; // Pin for Potentiometer (Analog input pin)

void setup() {
  pinMode(ledPin, OUTPUT); // Set the LED pin as OUTPUT
  Serial.begin(9600);
}

void loop() {
  int potValue = analogRead(potPin); // Read the value from the Potentiometer

  Serial.print("potValue : ");
  Serial.println(potValue);

  // Scale the potValue directly
```

```

int brightness = potValue / 4; // Scale down from 0-1023 to 0-255

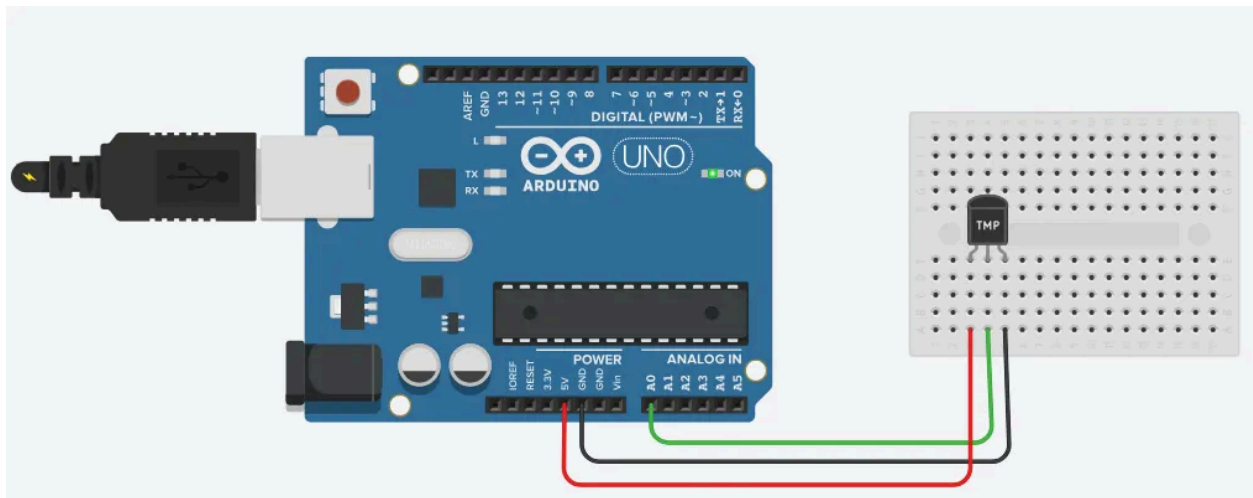
Serial.print("brightness : ");
Serial.println(brightness);

analogWrite(ledPin, brightness); // Write the brightness value to the LED
}

```

Temperature Sensor

ต่อวงจร Sensor อุณหภูมิโดยใช้ MCP9700 และเขียนโปรแกรมแสดงค่าอุณหภูมิที่วัดได้บน Serial Monitor



```

void setup() {
  Serial.begin(9600); // เริ่มต้น Serial Monitor
}

void loop() {
  int sensorValue = analogRead(A0); // อ่านค่าเซนเซอร์
  float voltage = sensorValue * (5.0 / 1023.0); // แปลงค่า Analog เป็น Voltage
  float temperatureC = voltage * 100; // แปลง Voltage เป็น อุณหภูมิ (Celsius)

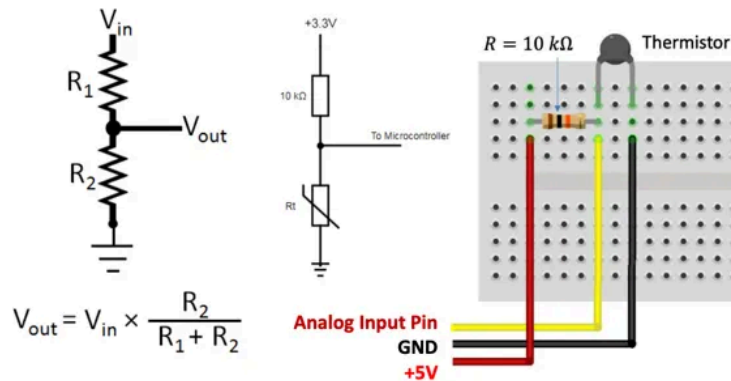
  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" C");

  delay(1000); // หน่วงเวลา 1 วินาที
}

```

Thermistor Sensor

ต่อวงจร Sensor อุณหภูมิโดยใช้ Thermistor และเขียนโปรแกรมแสดงค่าอุณหภูมิที่วัดได้บน Serial Monitor



ต่อวงจรตามรูปจะเห็นว่า
ค่าแรงดันที่เข้าขา **input** ของ
ADC จะมีค่า

จึงเขียนโปรแกรม แสดงค่าอุณหภูมิที่วัดได้ เทียบกับค่าที่ได้จาก **mcp9700**

Hint : แปลงแรงดัน เป็นค่าความต้านทาน แล้วแปลงไปเป็น อุณหภูมิ

```
// Thermistor parameters from the datasheet
#define RT0 10000
#define B 3977
```

```
// Our series resistor value = 330 Ω
#define R 330
```

```
// Variables for calculations
float RT, VR, In, TX, T0, VRT;
```

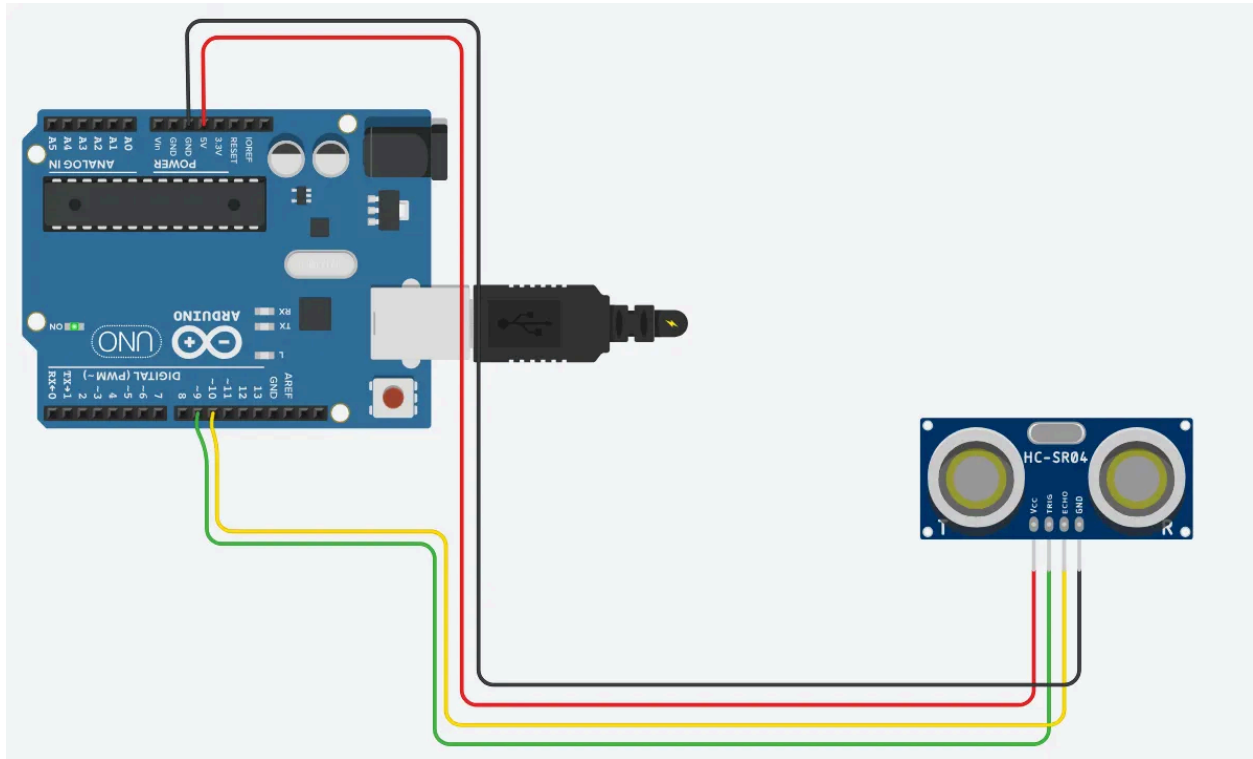
```
void setup() {
  // Setup serial communication
  Serial.begin(9600);
  // Convert T0 from Celsius to Kelvin
  T0 = 25 + 273.15;
}

void loop() {
  // Read the voltage across the thermistor
  VRT = (5.00 / 1023.00) * analogRead(A0);
  // Calculate the voltage across the resistor
  VR = 5.00 - VRT;
  // Calculate resistance of the thermistor
```

```
RT = VRT / (VR / R);  
// Calculate temperature from thermistor resistance  
ln = log(RT / RT0);  
TX = (1 / ((ln / B) + (1 / T0)));  
// Convert to Celsius  
TX = TX - 273.15;  
Serial.print("Temperature: ");  
// Display in Celsius  
Serial.print(TX);  
Serial.print("C\t");  
// Convert and display in Kelvin  
Serial.print(TX + 273.15);  
Serial.print("K\t");  
// Convert and display in Fahrenheit  
Serial.print((TX * 1.8) + 32);  
Serial.println("F");  
delay(500);  
}
```

Ultrasonic Sensor

โปรแกรมวัดระยะทาง แสดงผลออก Serial monitor



```
const int trigPin = 9;
const int echoPin = 10;
long duration;
int distanceCm, distanceInch;

void setup() {
  Serial.begin(9600); // Initialize serial communication at 9600 bits per second
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  // Clear the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Set the trigPin HIGH for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read the echoPin, return the sound wave travel time in microseconds
```

```
duration = pulseIn(echoPin, HIGH);

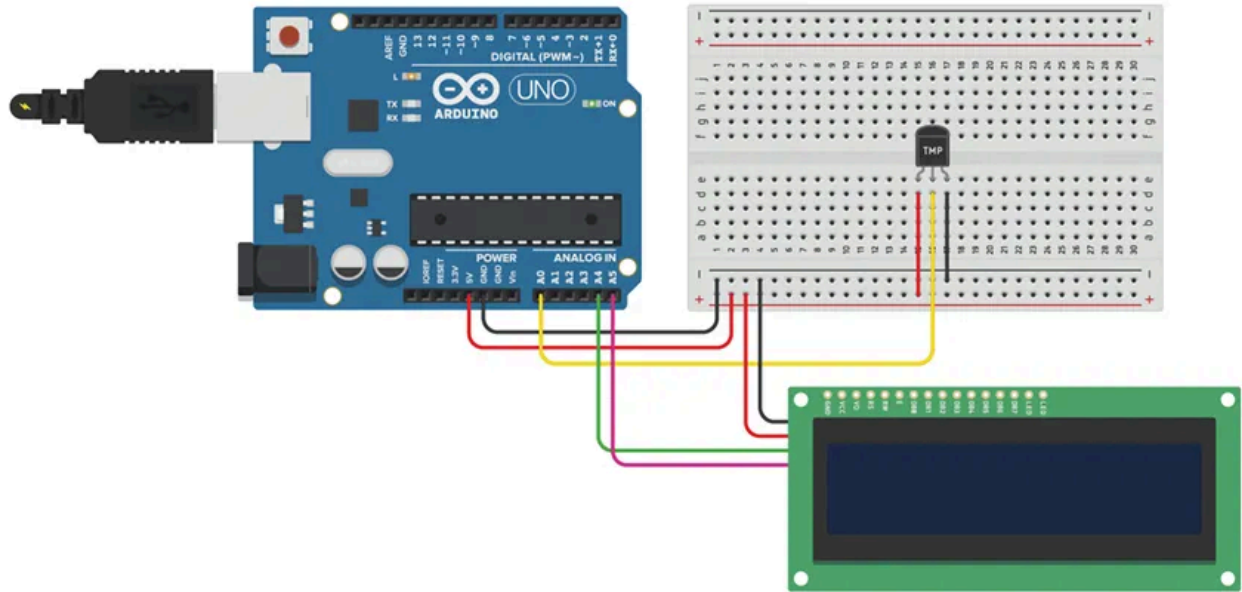
// Calculate the distance in cm and inches
distanceCm = duration * 0.034 / 2;
distanceInch = duration * 0.0133 / 2;

// Print the distances to the Serial Monitor
Serial.print("Distance: ");
Serial.print(distanceCm);
Serial.println(" cm");
    Serial.print(distanceInch);
    Serial.println(" in");

delay(1000); // Wait 1 second before the next measurement
}
```

Digital thermometer

เขียนโปรแกรมอ่านค่าอุณหภูมิจาก Sensor ไปแสดงผลออกที่จอ LCD แสดงค่าอุณหภูมิที่อ่านได้ (โดยมีทศนิยม 2 ตำแหน่ง)



```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  lcd.init();
  lcd.backlight();
}
void loop() {
  int sensorValue = analogRead(A0); // อ่านค่าเซนเซอร์
  float voltage = sensorValue * (5.0 / 1023.0); // แปลงค่า Analog เป็น Voltage
  float temperatureC = voltage * 100; // แปลง Voltage เป็น อุณหภูมิ (Celsius)
  lcd.setCursor(0,0);
  lcd.print(" "); // เคลียร์ตัวเลขเก่า
  lcd.setCursor(0,0);
  lcd.print(temperatureC, 2);
  lcd.print(" C");
  delay(1000);
}
```


Custom Font

จงสร้างตัวอักษรรูป ♥ จากนั้นให้แสดงออกจอ LCD โดยที่

-จอแถวแรก แสดงว่า I ♥ IT และ แสดงตรงกลางแถว

-จอแถวที่สองแสดงรหัสนักศึกษา และ ค่าอุณหภูมิ เช่น 25.54° (สร้าง custom font °)

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
byte customChar[] = {
```

```
  B00000,
```

```
  B00000,
```

```
  B01010,
```

```
  B11111,
```

```
  B11111,
```

```
  B01110,
```

```
  B00100,
```

```
  B00000
```

```
};
```

```
byte customChar2[] = {
```

```
  B11100,
```

```
  B10100,
```

```
  B11100,
```

```
  B00000,
```

```
  B00000,
```

```
  B00000,
```

```
  B00000,
```

```
  B00000
```

```
};
```

```
void setup() {
```

```
  lcd.init();
```

```
  lcd.backlight();
```

```
  lcd.createChar(1,customChar);
```

```
  lcd.setCursor(6,0);
```

```
  lcd.print("I");
```

```
  lcd.write(byte(1));
```

```
  lcd.print("IT");
```

```
}
```

```
void loop() {
```

```
  int sensorValue = analogRead(A0); // อ่านค่าเซนเซอร์
```

```
  float voltage = sensorValue * (5.0 / 1023.0); // แปลงค่า Analog เป็น Voltage
```

```
  float temperatureC = voltage * 100; // แปลง Voltage เป็น อุณหภูมิ (Celsius)
```

```
  float temperatureF = ((temperatureC * 9)/5)+32;
```

```
  lcd.createChar(2,customChar2);
```

```
lcd.setCursor(0,1);  
lcd.print("     ");// เคลียร์ตัวเลขเก่า  
lcd.setCursor(0,1);  
lcd.print(temperatureF, 2);  
lcd.write(byte(2));  
lcd.print("F ");  
lcd.print(temperatureC, 2);  
lcd.write(byte(2));  
lcd.print("C");  
delay(1000);  
}
```

อ่านค่าจาก ตัวต้านทานปรับค่าได้ แล้วแสดงค่า (0-1023) โดย Publish ไปที่ topic 67070134/light

```
/*
 *
 * This Arduino UNO R4 code is made available for public use without any restriction
 *
 */

#include <WiFi3.h>
#include <MQTTClient.h>

const char WIFI_SSID[] = ""; // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = ""; // CHANGE TO YOUR WIFI PASSWORD

const char MQTT_BROKER_ADDRESS[] = "phycom.it.kmitl.ac.th"; // CHANGE TO MQTT
BROKER'S ADDRESS
//const char MQTT_BROKER_ADDRESS[] = "192.168.0.11"; // CHANGE TO MQTT
BROKER'S IP ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "arduino-uno-r4-client"; // CHANGE IT AS YOU DESIRE
const char MQTT_USERNAME[] = ""; // CHANGE IT IF REQUIRED, empty if not
required
const char MQTT_PASSWORD[] = ""; // CHANGE IT IF REQUIRED, empty if not
required

// The MQTT topics that Arduino should publish/subscribe
const char PUBLISH_TOPIC[] = "67070134/light"; // CHANGE IT AS YOU DESIRE
const char SUBSCRIBE_TOPIC[] = "arduino-uno-r4/receive"; // CHANGE IT AS YOU DESIRE

const int PUBLISH_INTERVAL = 5000; // 5 seconds

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);

unsigned long lastPublishTime = 0;

void setup() {
  Serial.begin(9600);

  int status = WL_IDLE_STATUS;
  while (status != WL_CONNECTED) {
    Serial.print("Arduino UNO R4 - Attempting to connect to SSID: ");
    Serial.println(WIFI_SSID);
```

```

// Connect to WPA/WPA2 network. Change this line if using open or WEP network:
status = WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

// wait 10 seconds for connection:
delay(10000);
}
// print your board's IP address:
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

connectToMQTT();
}

void loop() {
  mqtt.loop();

  if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
    sendToMQTT();
    lastPublishTime = millis();
  }
}

void connectToMQTT() {
  // Connect to the MQTT broker
  mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

  // Create a handler for incoming messages
  mqtt.onMessage(messageReceived);

  Serial.print("Arduino UNO R4 - Connecting to MQTT broker");

  while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
    Serial.print(".");
    delay(100);
  }
  Serial.println();

  if (!mqtt.connected()) {
    Serial.println("Arduino UNO R4 - MQTT broker Timeout!");
    return;
  }

  // Subscribe to a topic, the incoming messages are processed by messageHandler() function
  if (mqtt.subscribe(SUBSCRIBE_TOPIC))

```

```

    Serial.print("Arduino UNO R4 - Subscribed to the topic: ");
else
    Serial.print("Arduino UNO R4 - Failed to subscribe to the topic: ");

    Serial.println(SUBSCRIBE_TOPIC);
    Serial.println("Arduino UNO R4 - MQTT broker Connected!");
}

void sendToMQTT() {

    // int val = millis()/1000;
    int val = analogRead(A0);
    String val_str = String(val);
    char messageBuffer[10];
    val_str.toCharArray(messageBuffer, 10);
    mqtt.publish(PUBLISH_TOPIC, messageBuffer);

    Serial.println("Arduino UNO R4 - sent to MQTT:");
    Serial.print("- topic: ");
    Serial.println(PUBLISH_TOPIC);
    Serial.print("- payload:");
    Serial.println(messageBuffer);
}

void messageReceived(String &topic, String &payload) {
    Serial.println("Arduino UNO R4 - received from MQTT:");
    Serial.println("- topic: " + topic);
    Serial.println("- payload:");
    Serial.println(payload);

    // You can process the incoming data , then control something
    /*
    process something
    */
}

```

อ่านค่าจาก Sensor อุณหภูมิ โดย Publish ไปที่ topic 67070134/temp

```
/*
 *
 * This Arduino UNO R4 code is made available for public use without any restriction
 *
 */

#include <WiFi3.h>
#include <MQTTClient.h>

const char WIFI_SSID[] = ""; // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = ""; // CHANGE TO YOUR WIFI PASSWORD

const char MQTT_BROKER_ADDRESS[] = "phycom.it.kmitl.ac.th"; // CHANGE TO MQTT
BROKER'S ADDRESS
//const char MQTT_BROKER_ADDRESS[] = "192.168.0.11"; // CHANGE TO MQTT
BROKER'S IP ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "arduino-uno-r4-client"; // CHANGE IT AS YOU DESIRE
const char MQTT_USERNAME[] = ""; // CHANGE IT IF REQUIRED, empty if not
required
const char MQTT_PASSWORD[] = ""; // CHANGE IT IF REQUIRED, empty if not
required

// The MQTT topics that Arduino should publish/subscribe
const char PUBLISH_TOPIC[] = "67070134/temp"; // CHANGE IT AS YOU DESIRE
const char SUBSCRIBE_TOPIC[] = "arduino-uno-r4/receive"; // CHANGE IT AS YOU DESIRE

const int PUBLISH_INTERVAL = 5000; // 5 seconds

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);

unsigned long lastPublishTime = 0;

void setup() {
  Serial.begin(9600);

  int status = WL_IDLE_STATUS;
  while (status != WL_CONNECTED) {
    Serial.print("Arduino UNO R4 - Attempting to connect to SSID: ");
    Serial.println(WIFI_SSID);
    // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
    status = WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  }
}
```

```

    // wait 10 seconds for connection:
    delay(10000);
}
// print your board's IP address:
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

connectToMQTT();
}

void loop() {
    mqtt.loop();

    if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
        sendToMQTT();
        lastPublishTime = millis();
    }
}

void connectToMQTT() {
    // Connect to the MQTT broker
    mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

    // Create a handler for incoming messages
    mqtt.onMessage(messageReceived);

    Serial.print("Arduino UNO R4 - Connecting to MQTT broker");

    while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    if (!mqtt.connected()) {
        Serial.println("Arduino UNO R4 - MQTT broker Timeout!");
        return;
    }

    // Subscribe to a topic, the incoming messages are processed by messageHandler() function
    if (mqtt.subscribe(SUBSCRIBE_TOPIC))
        Serial.print("Arduino UNO R4 - Subscribed to the topic: ");
    else

```

```

    Serial.print("Arduino UNO R4 - Failed to subscribe to the topic: ");

    Serial.println(SUBSCRIBE_TOPIC);
    Serial.println("Arduino UNO R4 - MQTT broker Connected!");
}

void sendToMQTT() {

    // int val = millis()/1000;
    int sensorValue = analogRead(A0);
    float voltage = sensorValue * (5.0 / 1023.0);
    float temperatureC = voltage * 100;
    String val_str = String(temperatureC, 2);
    Serial.print("Raw = ");
    Serial.print(sensorValue);
    Serial.print(" Voltage = ");
    Serial.print(voltage);
    Serial.println(" V");
    char messageBuffer[16];
    val_str.toCharArray(messageBuffer, 16);
    mqtt.publish(PUBLISH_TOPIC, messageBuffer);

    Serial.println("Arduino UNO R4 - sent to MQTT:");
    Serial.print("- topic: ");
    Serial.println(PUBLISH_TOPIC);
    Serial.print("- payload:");
    Serial.println(messageBuffer);
}

void messageReceived(String &topic, String &payload) {
    Serial.println("Arduino UNO R4 - received from MQTT:");
    Serial.println("- topic: " + topic);
    Serial.println("- payload:");
    Serial.println(payload);

    // You can process the incoming data , then control something
    /*
    process something
    */
}

```


**อ่านค่าระยะทางจาก Ultrasonic หน่วยเป็น cm โดย ถ้ามากกว่า 20 cm
ให้ Publish คำว่า off ไปที่ topic 67070134/food**

```
/*
 *
 * This Arduino UNO R4 code is made available for public use without any restriction
 *
 */

#include <WiFi3.h>
#include <MQTTClient.h>

const char WIFI_SSID[] = "PATOMassakorn"; // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "0937842332"; // CHANGE TO YOUR WIFI PASSWORD

const char MQTT_BROKER_ADDRESS[] = "phycom.it.kmitl.ac.th"; // CHANGE TO MQTT
BROKER'S ADDRESS
//const char MQTT_BROKER_ADDRESS[] = "192.168.0.11"; // CHANGE TO MQTT
BROKER'S IP ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "arduino-uno-r4-client"; // CHANGE IT AS YOU DESIRE
const char MQTT_USERNAME[] = ""; // CHANGE IT IF REQUIRED, empty if not
required
const char MQTT_PASSWORD[] = ""; // CHANGE IT IF REQUIRED, empty if not
required

// The MQTT topics that Arduino should publish/subscribe
const char PUBLISH_TOPIC[] = "67070134/food"; // CHANGE IT AS YOU DESIRE
const char SUBSCRIBE_TOPIC[] = "arduino-uno-r4/receive"; // CHANGE IT AS YOU DESIRE

const int PUBLISH_INTERVAL = 5000; // 5 seconds

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);

unsigned long lastPublishTime = 0;

const int trigPin = 9;
const int echoPin = 10;
long duration;
int distanceCm, distanceInch;

void setup() {
  Serial.begin(9600);
```

```

int status = WL_IDLE_STATUS;
while (status != WL_CONNECTED) {
  Serial.print("Arduino UNO R4 - Attempting to connect to SSID: ");
  Serial.println(WIFI_SSID);
  // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
  status = WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  // wait 10 seconds for connection:
  delay(10000);
}
// print your board's IP address:
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

connectToMQTT();

pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
}

void loop() {
  mqtt.loop();

  if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
    sendToMQTT();
    lastPublishTime = millis();
  }
}

void connectToMQTT() {
  // Connect to the MQTT broker
  mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

  // Create a handler for incoming messages
  mqtt.onMessage(messageReceived);

  Serial.print("Arduino UNO R4 - Connecting to MQTT broker");

  while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
    Serial.print(".");
    delay(100);
  }
  Serial.println();
}

```

```

if (!mqtt.connected()) {
  Serial.println("Arduino UNO R4 - MQTT broker Timeout!");
  return;
}

// Subscribe to a topic, the incoming messages are processed by messageHandler() function
if (mqtt.subscribe(SUBSCRIBE_TOPIC))
  Serial.print("Arduino UNO R4 - Subscribed to the topic: ");
else
  Serial.print("Arduino UNO R4 - Failed to subscribe to the topic: ");

Serial.println(SUBSCRIBE_TOPIC);
Serial.println("Arduino UNO R4 - MQTT broker Connected!");
}

void sendToMQTT() {

  // int val = millis()/1000;
  char messageBuffer[16];
  // Clear the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Set the trigPin HIGH for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Read the echoPin, return the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  // Calculate the distance in cm and inches
  distanceCm = duration * 0.034 / 2;
  distanceInch = duration * 0.0133 / 2;
  if(distanceCm > 20){
    strcpy(messageBuffer, "off");
  }else{
    strcpy(messageBuffer, "on");
  }
  mqtt.publish(PUBLISH_TOPIC, messageBuffer);

  Serial.println("Arduino UNO R4 - sent to MQTT:");
  Serial.print("- topic: ");

```

```
Serial.println(PUBLISH_TOPIC);
Serial.print("- payload:");
Serial.println(messageBuffer);
}

void messageReceived(String &topic, String &payload) {
  Serial.println("Arduino UNO R4 - received from MQTT:");
  Serial.println("- topic: " + topic);
  Serial.println("- payload:");
  Serial.println(payload);

  // You can process the incoming data , then control something
  /*
  process something
  */
}
```

Subscribe ค่าจาก Topic 67070134/venus นำค่าที่ได้รับได้ แสดงผลออกทาง Color LED ดังนี้ หากค่าตั้งแต่ 36 - 50 ให้ LED เป็นสีแดง หากค่าตั้งแต่ 26 - 35 ให้ LED เป็นสีฟ้า หากค่าตั้งแต่ 10 - 25 ให้ LED เป็นสีเขียว

```
/*
 *
 * This Arduino UNO R4 code is made available for public use without any restriction
 *
 */

#include <WiFi3.h>
#include <MQTTClient.h>

const char WIFI_SSID[] = "ATOMPassakorn"; // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "0937842332"; // CHANGE TO YOUR WIFI PASSWORD

const char MQTT_BROKER_ADDRESS[] = "phycom.it.kmitl.ac.th"; // CHANGE TO MQTT
BROKER'S ADDRESS
//const char MQTT_BROKER_ADDRESS[] = "192.168.0.11"; // CHANGE TO MQTT
BROKER'S IP ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "arduino-uno-r4-client"; // CHANGE IT AS YOU DESIRE
const char MQTT_USERNAME[] = ""; // CHANGE IT IF REQUIRED, empty if not
required
const char MQTT_PASSWORD[] = ""; // CHANGE IT IF REQUIRED, empty if not
required

// The MQTT topics that Arduino should publish/subscribe
const char PUBLISH_TOPIC[] = "arduino-uno-r4/send"; // CHANGE IT AS YOU DESIRE
const char SUBSCRIBE_TOPIC[] = "67070134/venus"; // CHANGE IT AS YOU DESIRE

const int PUBLISH_INTERVAL = 5000; // 5 seconds

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);

unsigned long lastPublishTime = 0;

int ledPin9 = 9;
int ledPin10 = 10;
int ledPin11 = 11;

void setup() {
```

```

Serial.begin(9600);

int status = WL_IDLE_STATUS;
while (status != WL_CONNECTED) {
  Serial.print("Arduino UNO R4 - Attempting to connect to SSID: ");
  Serial.println(WIFI_SSID);
  // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
  status = WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  // wait 10 seconds for connection:
  delay(10000);
}
// print your board's IP address:
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

connectToMQTT();

pinMode(ledPin9, OUTPUT);
pinMode(ledPin10, OUTPUT);
pinMode(ledPin11, OUTPUT);
}

void loop() {
  mqtt.loop();

  if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
    sendToMQTT();
    lastPublishTime = millis();
  }
}

void connectToMQTT() {
  // Connect to the MQTT broker
  mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

  // Create a handler for incoming messages
  mqtt.onMessage(messageReceived);

  Serial.print("Arduino UNO R4 - Connecting to MQTT broker");

  while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
    Serial.print(".");
    delay(100);
  }
}

```

```

}
Serial.println();

if (!mqtt.connected()) {
    Serial.println("Arduino UNO R4 - MQTT broker Timeout!");
    return;
}

// Subscribe to a topic, the incoming messages are processed by messageHandler() function
if (mqtt.subscribe(SUBSCRIBE_TOPIC))
    Serial.print("Arduino UNO R4 - Subscribed to the topic: ");
else
    Serial.print("Arduino UNO R4 - Failed to subscribe to the topic: ");

Serial.println(SUBSCRIBE_TOPIC);
Serial.println("Arduino UNO R4 - MQTT broker Connected!");
}

void sendToMQTT() {

    int val = millis()/1000;
    //int val = analogRead(A0);
    String val_str = String(val);
    char messageBuffer[10];
    val_str.toCharArray(messageBuffer, 10);
    mqtt.publish(PUBLISH_TOPIC, messageBuffer);

    Serial.println("Arduino UNO R4 - sent to MQTT:");
    Serial.print("- topic: ");
    Serial.println(PUBLISH_TOPIC);
    Serial.print("- payload:");
    Serial.println(messageBuffer);
}

void messageReceived(String &topic, String &payload) {
    Serial.println("Arduino UNO R4 - received from MQTT:");
    Serial.println("- topic: " + topic);
    Serial.println("- payload:");
    Serial.println(payload);

    // You can process the incoming data , then control something
    /*
    process something
    */
}

```

```
int value = payload.toInt();
digitalWrite(ledPin9, HIGH);
digitalWrite(ledPin10, HIGH);
digitalWrite(ledPin11, HIGH);
if(value>=36 && value<=50){
    digitalWrite(ledPin9, LOW);
    digitalWrite(ledPin10, HIGH);
    digitalWrite(ledPin11, HIGH);
}else if(value>=26 && value<=35){
    digitalWrite(ledPin9, HIGH);
    digitalWrite(ledPin10, HIGH);
    digitalWrite(ledPin11, LOW);
}else if(value>=10 && value<=25){
    digitalWrite(ledPin9, HIGH);
    digitalWrite(ledPin10, LOW);
    digitalWrite(ledPin11, HIGH);
}
}
```